

CS304: PROGRAMMING ON THE WEB

DR. DALIA RIZK



جامعة الأهرام الكندية
AHRAM CANADIAN UNIVERSITY

DESIGN OF COMPLEX WEB-BASED PROGRAMS

□ Definition

□ Features

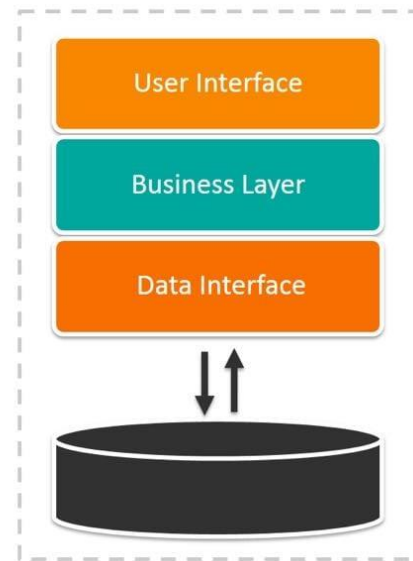
DESIGN OF COMPLEX WEB-BASED PROGRAMS

- ❑ Designing complex web-based programs (e.g., SaaS platforms, e-commerce engines, health management systems) requires a rigorous approach beyond standard website creation, focusing on **scalability, security, user experience, and robust functionality.**

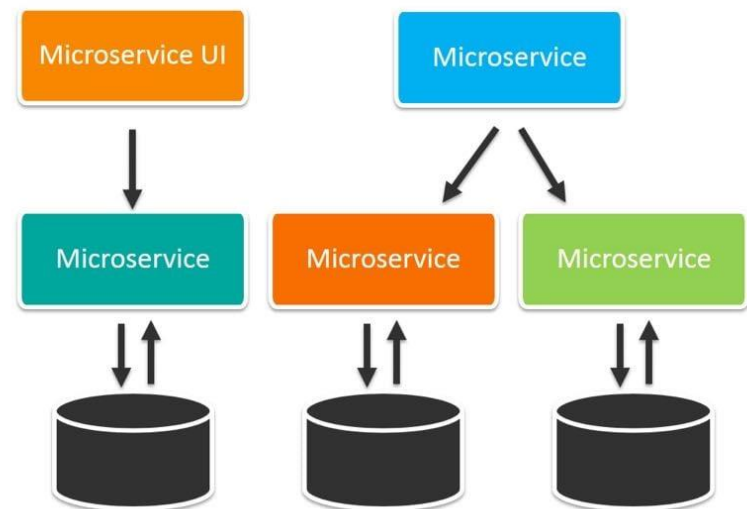
ARCHITECTURAL EVOLUTION

- ❑ **Monolithic Architecture:** The entire application is built as a single, unified unit. Simpler to develop and deploy initially, but harder to scale and maintain as it grows
- ❑ **Microservices Architecture:** The application is divided into small, independent services that communicate via APIs. Highly scalable and flexible

Monolithic Architecture



Microservices Architecture



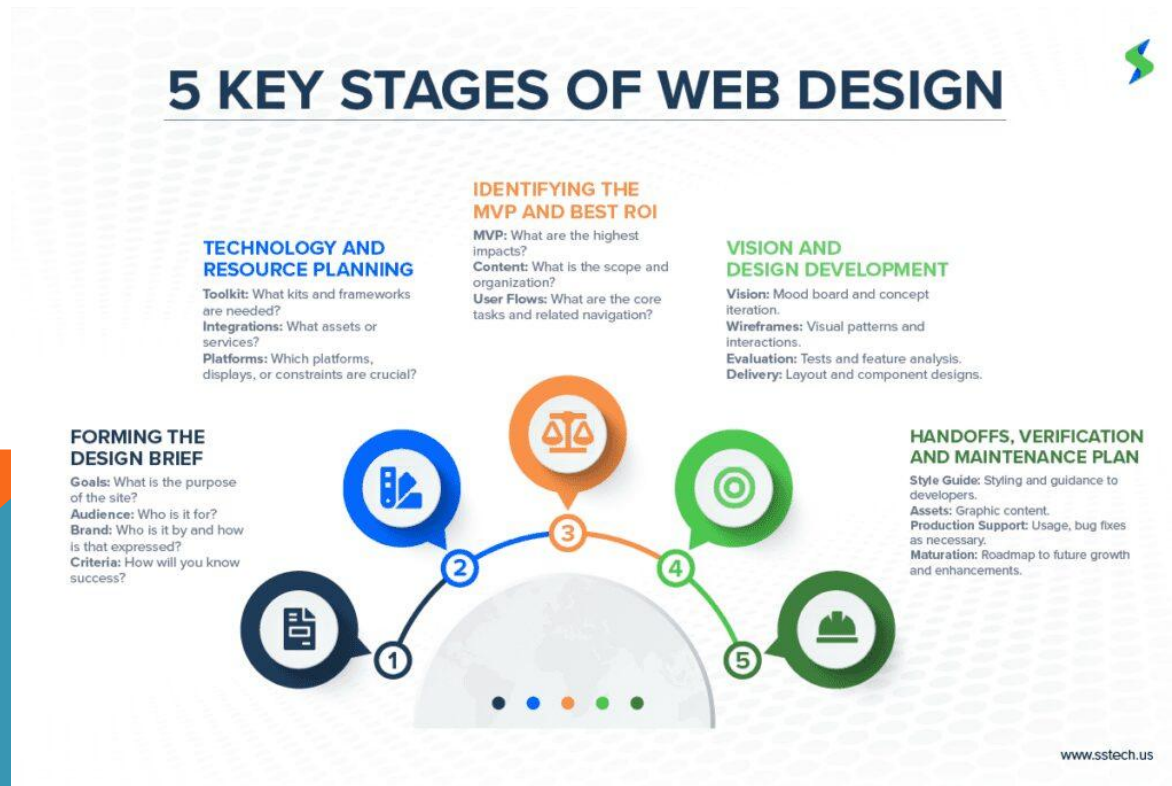
MICROSERVICES DESIGN PRINCIPLES

- ❑ One service does one thing well.
- ❑ Services are independent.
- ❑ Use different languages/DBs per service.
- ❑ Communication happens via standard protocols.



KEY ASPECTS OF COMPLEX WEB DESIGN

- ❑ **Comprehensive Research:** successful projects start with market and user research (e.g., using tools like Similarweb) to identify user pain points, competitor gaps, and necessary features
- ❑ **Structured Planning:** defining the scope, objectives, and milestones using SMART goals is critical to prevent scope creep



KEY ASPECTS OF COMPLEX WEB DESIGN – CONT'D

- ❑ **Technical Architecture:** choosing the right tech stack (e.g., using Wappalyzer to analyze competitor tools) is crucial, as is balancing front-end user interface (UI) with back-end database and logic
- ❑ **Prototyping & Testing:** Developing high-fidelity prototypes (e.g., using Figma or Adobe XD) and conducting user testing
- ❑ **Dynamic Functionality:** Complex sites often require dynamic web design, featuring interactive content, databases, and APIs, rather than simple, static layouts

DOMAIN – DRIVEN DESIGN (DDD)

- ❑ To bridge the gap between technical team and domain expert, it is required to:
 - Define a clear boundaries where a particular model is defined and applicable
 - Use a common language by everyone to talk about the domain
 - Use clusters of domain objects that can be treated as a single unit
 - Decide on type of scaling strategy:
 - **Vertical Scaling** (Scaling Up): Adding more power (CPU, RAM) to an existing machine. Limited by hardware capacity. Slower recovery from failure.
 - **Horizontal Scaling** (Scaling Out): Adding more machines to your network. Virtually unlimited scale. High availability and resilience.

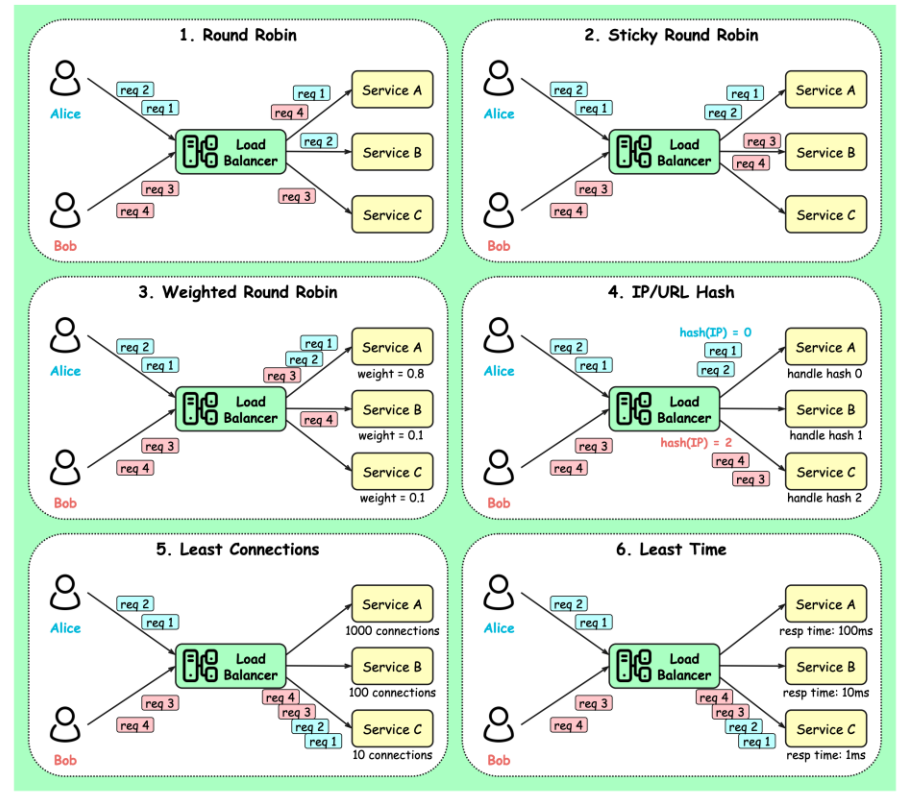
ALGORITHMS

Load balancing algorithms:

- **Round Robin:** Requests are distributed sequentially
- **Least Connections:** Sends requests to server with fewest active jobs
- **IP Hash:** Maps client IP to a specific server (Session persistence)

Load Balancing Algorithms

blog.bytebytego.com



DATABASES

❑ Relational vs. Non-Relational

Type	Relational (SQL)	Non-Relational (NoSQL)
Structure	Strict Schema (Tables/Rows)	Dynamic Schema (Key-Value, Doc)
Scaling	Mostly Vertical	Horizontal
Consistency	ACID Compliant	BASE (Eventual Consistency)

❑ Database scaling and optimization: by using performance techniques:

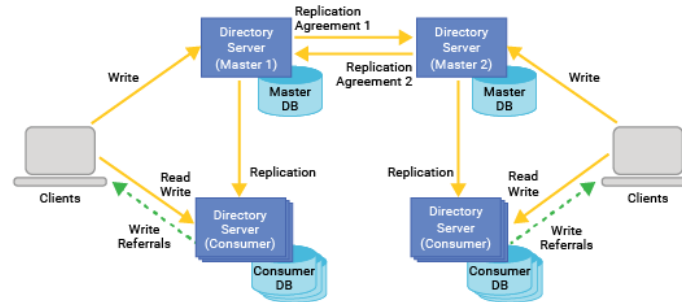
- **Indexing:** Creating data structures for rapid lookup
- **Read Replicas:** Offloading read traffic to auxiliary databases
- **Normalization:** Reducing data redundancy

DATABASES – CONT'D

- ❑ **Horizontal Partitioning:** Sharding breaks a large database into smaller, faster, more easily managed parts called **shards**. Each shard is held on a separate database server instance, spreading the load and improving performance for massive datasets.

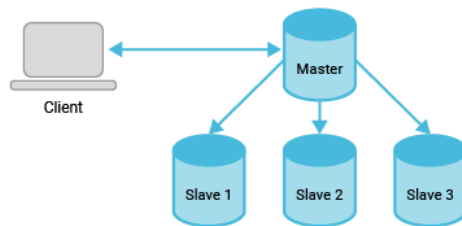
Multi-Master Replication

based off the DynamoDB model



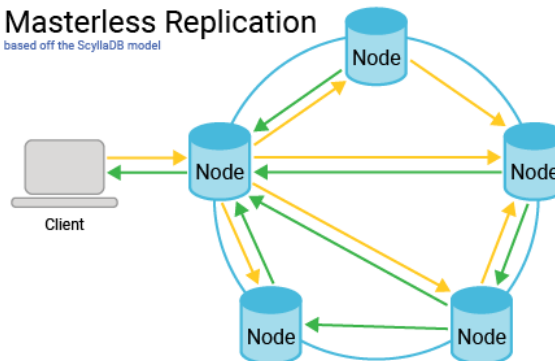
Master-Slave Replication

based off the MongoDB model



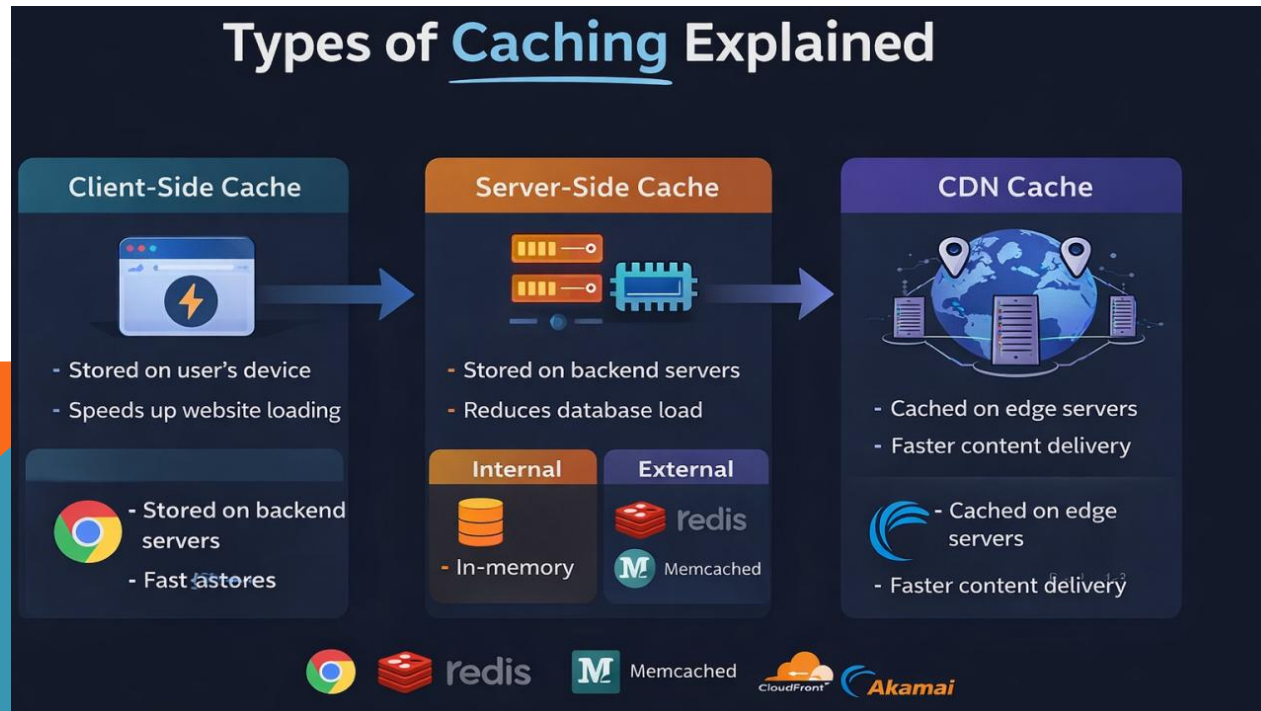
Masterless Replication

based off the ScyllaDB model



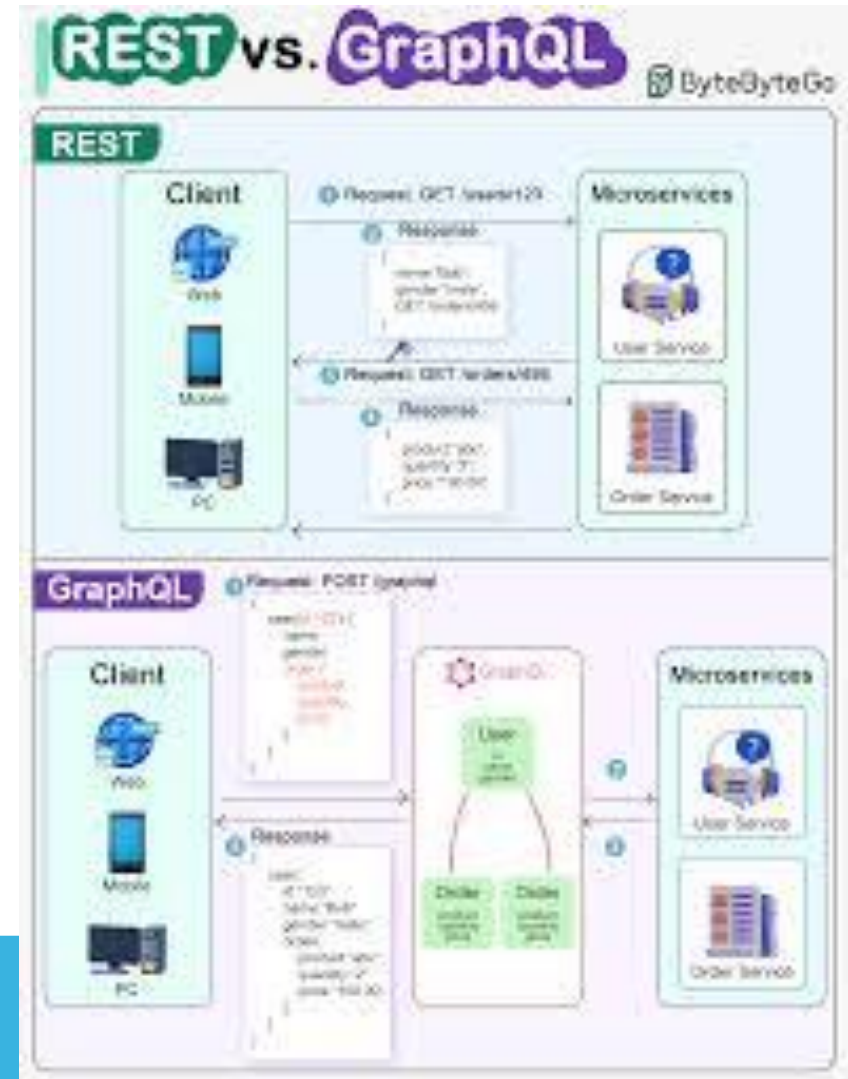
CACHING

- ❑ **CDN Caching:** Caching static assets (JS, CSS, Images) closer to the user geographically
- ❑ **In-Memory DB:** Using Redis or Memcached for lightning-fast key-value storage of frequent queries
- ❑ **Client-Side:** Browser cache and Service Workers for offline capabilities



API

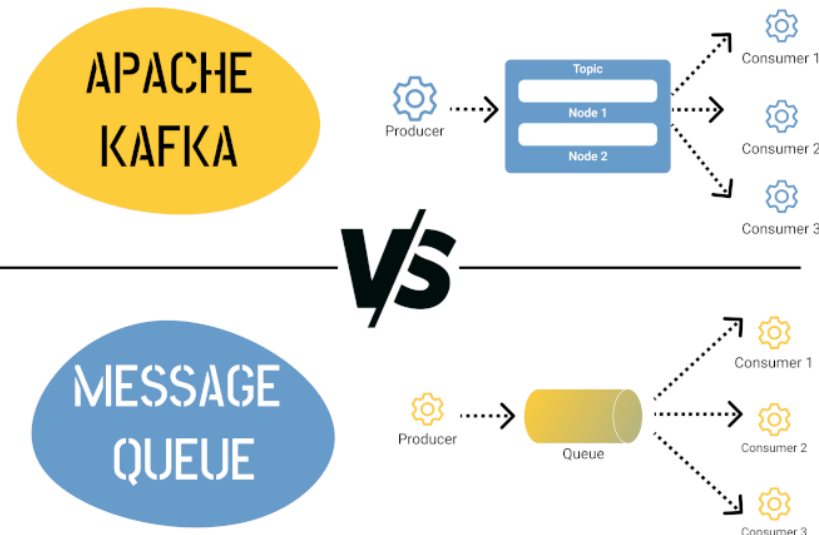
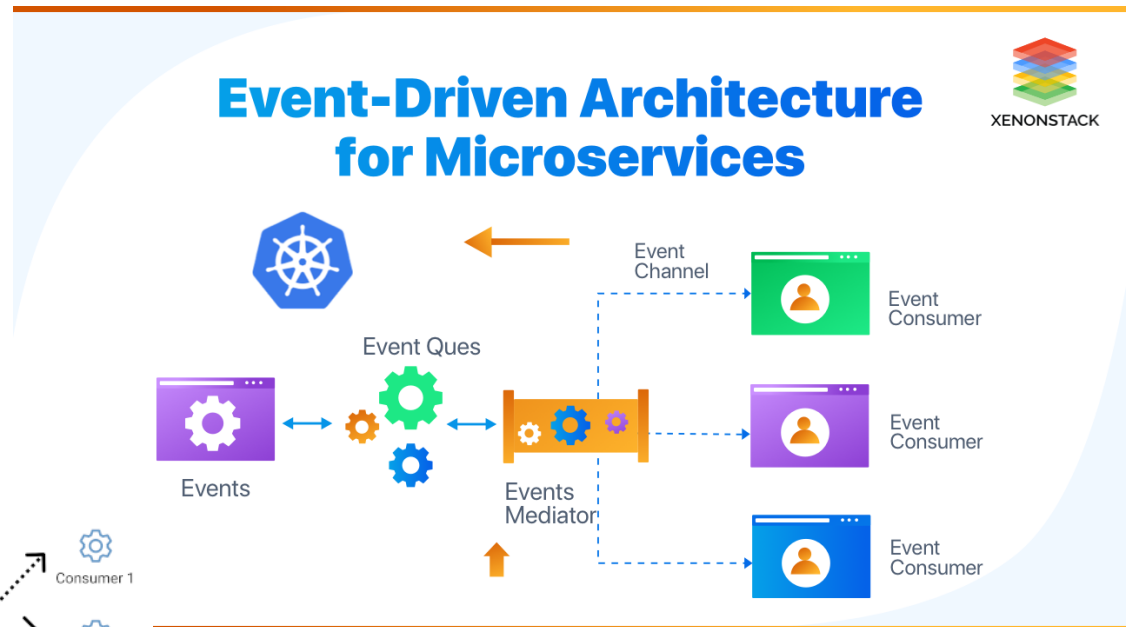
- ❑ **REST (Representational State Transfer):** Resource-based, multiple endpoints, over-fetching possible
- ❑ **GraphQL:** Single endpoint, client defines requirements, no over-fetching



ASYNCHRONOUS MESSAGING SYSTEM (FLOW)

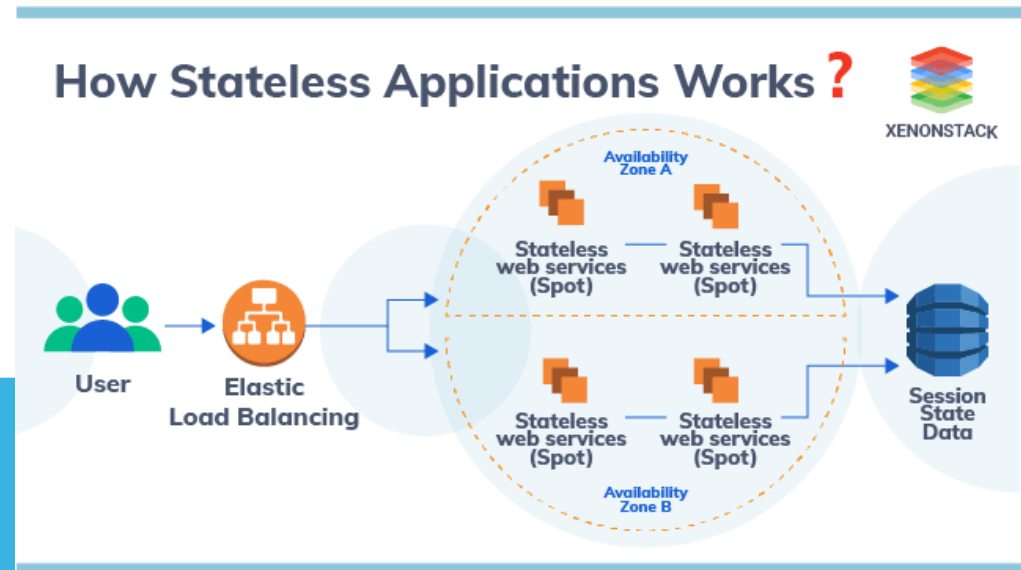
- ❑ Event-driven architecture allows services to react to changes (events) instead of waiting for direct calls

- Decouples services
- Handles traffic spikes
- Guaranteed delivery



MANAGING STATE

- ❑ In a complex web system, application servers should be stateless
- Session data should reside in a centralized, high-speed store (example: Redis).
- Resilience to individual server failure
- Easier horizontal scaling
- Simplified load balancing



CAP THEOREM

- ❑ In a distributed data store, you can only provide two of the following **three** guarantees simultaneously:

Component	Description
Consistency	Every read receives the most recent write or an error.
Availability	Every request receives a (non-error) response.
Partition Tolerance	The system continues to operate despite network failures.

HIGH AVAILABILITY

- ❑ **Redundancy:** Eliminating single points of failure by duplicating critical components
- ❑ **Failover:** Automatic switching to a redundant or standby computer server/network upon failure
- ❑ **RTO & RPO:** Recovery Time Objective (Downtime) and Recovery Point Objective (Data loss)

